

Ergebnisbericht: Abgabe 3 - Betriebssysteme VL Uni Trier WS 25/26 - Marvin Mogleke - 1779643

Generelles

Zur Arbeit mit dem ZFS-Dateisystem hab ich mithilfe von Linux' `virt-manager` eine virtuelle Maschine mit dem Ubuntu Server 24.04.4-Image erstellt. Über SSH habe ich mich dann von meinem Host-Systems mit der VM verbunden und die notwendigen Pakete installiert sowie Dateien übertragen.

Aufgabe 1

Zur Erstellung von Sicherungen hab ich in Zusammenarbeit mit der KI meines Vertrauens ein Python-Skript geschrieben, welches die ZFS-Befehle über `subprocess.run` ausführt. Folgende Parameter können über eine `config.json` angepasst werden: `source_dataset`, `backup_location`, `max_backups`, `snapshot_prefix` und `pool_name`. Ich habe hier direkt ab Aufgabe 1 mit einem ZFS-Pool gearbeitet; dafür habe ich der VM insgesamt 4 weitere 5-GB-Platten hinzugefügt und mit `zpool create backuppool raidz /dev/vdb /dev/vdc /dev/vdd /dev/vde` einen Pool erstellt.

Bei Ausführung des Skripts erstellt es automatisch Snapshots des in der `config.json` angegebenen Datasets, überträgt diese in das Backup-Verzeichnis und verwaltet die Anzahl der Sicherungen entsprechend der Konfiguration. Es bietet auch die Möglichkeit, Sicherungen wiederherzustellen und eine Liste aller vorhandenen Snapshots anzuzeigen.

Das Skript kann mit folgenden Flags ausgeführt werden:

```
usage: zfs_backup.py [-h] [--config CONFIG] [--create-config] [--status] [--restore FILE] [--list-snapshots]
```

ZFS Backup and Archive Tool

options:

<code>-h, --help</code>	show this <code>help</code> message and <code>exit</code>
<code>--config CONFIG</code>	Path to config file
<code>--create-config</code>	Create sample config
<code>--status</code>	Show status
<code>--restore FILE</code>	Restore from backup file
<code>--list-snapshots</code>	List snapshots

Testweise habe ich eine Textdatei in dem zu sichernden Dataset erstellt und nach der Sicherung gelöscht. Nach der Wiederherstellung war die Datei wieder vorhanden.

```

marvin@zsf-test-vm:~$ sudo sh -c 'echo "I love ZFS" > /backuppool/dataset/zfs_love.txt'
marvin@zsf-test-vm:~$ sudo ./zfs_backup.py
=====
Starting ZFS Backup
=====
Creating snapshot: backuppool/dataset@backup_20260312_105221
Running: zfs snapshot backuppool/dataset@backup_20260312_105221
Sending snapshot backuppool/dataset@backup_20260312_105221 to /backup/backup_20260312_105221.zfs
Cleaning up snapshot: backuppool/dataset@backup_20260312_105221
Running: zfs destroy backuppool/dataset@backup_20260312_105221
=====
Backup completed successfully!
Backup file: /backup/backup_20260312_105221.zfs
Total backups: 1
=====
marvin@zsf-test-vm:~$ sudo ./zfs_backup.py --status
ZFS Backup Tool Status
=====
Source Dataset: backuppool/dataset
Backup Location: /backup
Max Backups: 5
Snapshot Prefix: backup_
Pool Name: backuppool

Current Backups: 1
- backup_20260312_105221.zfs (0.04 MB) [ ]

Running: zfs list -t snapshot -r backuppool/dataset
Existing snapshots:

marvin@zsf-test-vm:~$ sudo rm /backuppool/dataset/zfs_love.txt
marvin@zsf-test-vm:~$ cat /backuppool/dataset/zfs_love.txt
cat: /backuppool/dataset/zfs_love.txt: No such file or directory

```

```

marvin@zsf-test-vm:~$ sudo ./zfs_backup.py --restore /backup/backup_20260312_105221.zfs
Restoring from: /backup/backup_20260312_105221.zfs
marvin@zsf-test-vm:~$ cat /backuppool/dataset/zfs_love.txt
I love ZFS
marvin@zsf-test-vm:~$

```

Aufgabe 2

Um eine Bearbeitung während der Snapshot Erstellung zu simulieren, habe ich ein weiteres kleines Hilfsscript geschrieben: `time\_logger.p`. Dieses Script schreibt alle 2 Sekunden die aktuelle Zeit in eine neue Zeile einer Textdatei. Dann wartet es 2 Sekunden, bevor es die Zeile als vollständig markiert. Ich habe dieses Script über ein separates Terminal auf der VM laufen lassen, während ich eine Sicherung erstellt habe. Danach habe ich die originale Textdatei mit der im Snapshot gesicherten Version verglichen.

```

marvin@zsf-test-vm:~$ cat /backuppool/dataset/.zfs/snapshot/zeit_test/uhrzeit.txt | tail -n 5; echo
Uhrzeit: 11:20:38 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:40 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:42 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:44 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:46 - [Snapshot-Fenster offen]...
marvin@zsf-test-vm:~$ cat /backuppool/dataset/uhrzeit.txt | tail -n 5; echo
Uhrzeit: 11:20:46 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:48 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:50 - [Snapshot-Fenster offen]... Zeile komplett!
Uhrzeit: 11:20:52 - [Snapshot-Fenster offen]... Zeile komplett!

```

Auf dem Bild sieht man, dass der Snapshot nur bis zum Sicherungszeitpunkt geschriebenen Zeiten enthält, während die Zeilen, die nach der Sicherung geschrieben wurden, nicht in der Sicherung enthalten sind. In dem simulierten Beispielfall mit einer Textdatei führt ein ZFS-Snapshot während eines laufenden Schreibvorgangs lediglich zu einem unvollständigen Datenanhang – beispielsweise einem abgeschnittenen Satz oder einer halben Zeile. Die Datei als Ganzes bleibt funktional und kann nach einer Wiederherstellung weiterhin problemlos in einem Editor geöffnet und gelesen werden.

Komplexe Applikationen wie Datenbanken oder virtuelle Maschinen setzen hingegen eine strikte strukturelle und logische Integrität voraus. Wird bei solchen Systemen eine zusammenhängende Transaktion durch den exakten Zeitpunkt des Snapshots genau in der Mitte durchtrennt ("Torn Write"), ist die resultierende Sicherungsdatei in sich logisch defekt. Da der Anwendung beim späteren Einlesen essenzielle Puzzlesteine fehlen, stuft sie das Backup als korrupt ein und verweigert den Dienst oder stürzt ab.

Verhindern könnte man das Problem durch Einfrieren solcher Applikationen während der Snapshot-Erstellung. So könnte man vor der Sicherung mithilfe von Skripten Anwendungen anweisen, alle Daten auf die Festplatte zu schreiben und den Zugriff auf die Dateien zu sperren, bis der Snapshot abgeschlossen ist. Alternativ verfügen moderne relationale Datenbanken über integrierte Transaktionsprotokolle (Write-Ahead Logging). Wird ein logisch inkonsistenter ZFS-Snapshot wiederhergestellt, erkennt die Datenbank beim Start anhand ihres internen Protokolls, dass eine Transaktion durch den Moment des Snapshots abgerissen ist. Sie

nutzt dann diese Log-Dateien, um den unvollständigen Vorgang selbstständig und sauber zurückzurollen (Rollback), wodurch die logische Integrität wiederhergestellt wird.

Aufgabe 3

Zum Test habe ich wieder das `time_logger` -Skript aus Aufgabe 2 verwendet. Ich habe das Skript gestartet, während ich in einer anderen Terminalinstanz den Status des ZFS-Pools mit `watch -n 1 sudo zpool status` überwacht habe. Dann habe ich eine der Platten des Pools in der VM-Konfiguration entfernt. Im Statusüberwachungsfenster konnte ich sofort sehen, wie der Pool in den `DEGRADED` -Zustand wechselt, während das Skript einfach weitergelaufen ist.

```
marvin@zsf-test-vm:~$
marvin@zsf-test-vm:~$
marvin@zsf-test-vm:~$
marvin@zsf-test-vm:~$
marvin@zsf-test-vm:~$
marvin@zsf-test-vm:~$
marvin@zsf-test-vm:~$ sudo python3 time_logger.py
[sudo] password for marvin:
Schreibe Sekunden-Log in /backuppools/dataset/uhrzeit.txt..
.
^CTraceback (most recent call last):
  File "/home/marvin/time_logger.py", line 15, in <module>
    time.sleep(2)
KeyboardInterrupt

marvin@zsf-test-vm:~$ sudo python3 time_logger.py
Schreibe Sekunden-Log in /backuppools/dataset/uhrzeit.txt..
.

Every 1.0s: sudo... zsf-test-vm: Thu Mar 12 11:51:34 2026

pool: backuppools
state: DEGRADED
status: One or more devices has been removed by the administrator.
        Sufficient replicas exist for the pool to continue
        functioning in a degraded state.
action: Online the device using 'zpool online' or replace the device with
        'zpool replace'.
config:

    NAME        STATE      READ  WRITE CKSUM
    backuppools  DEGRADED   0     0     0
    raidz1-0     DEGRADED   0     0     0
    vdb          REMOVED    0     0     0
    vdc          ONLINE     0     0     0
    vdd          ONLINE     0     0     0
```

Aufgabe 4

Um ein ausgeglichenes Spielfeld zu schaffen, habe ich zwei komplett neue VMs mit identischer Konfiguration erstellt (4 GB RAM, 2 Kerne, 20 GB Platte). Auf der ZFS-VM habe ich einen neuen Pool mit einer 20-GB-Platte erstellt. Auf der ext4-VM habe ich die 20-GB-Platte mit `mkfs.ext4` formatiert und in das Dateisystem eingebunden. Zum Benchmarking habe ich das `fio` Tool verwendet, welches ich auf beiden VMs installiert habe.

In meinem `run_benchmark` -Skript habe ich zwei Testszenarien konfiguriert. Das Skript simuliert zwei verschiedene, praxisnahe Belastungsszenarien, um sowohl den maximalen Durchsatz (Bandbreite in MB/s) als auch die Verarbeitungsgeschwindigkeit von kleinen Datenblöcken (IOPS) zu messen.

Test 1: Sequentielles Schreiben (Fokus auf Durchsatz)

Dieser Test simuliert das Speichern großer, zusammenhängender Dateien.

- `--rw=write` : Definiert ein rein sequentielles Schreibmuster. Die Daten werden linear und ohne Sprünge auf den Datenträger geschrieben.
- `--bs=1M` : Setzt die Blockgröße auf 1 Megabyte. Dies ist ideal, um die maximale Bandbreite der Festplatte und des Dateisystems auszureizen.
- `--size=1G` : Es wird insgesamt eine Datenmenge von 1 Gigabyte geschrieben, um den Cache des Systems zu füllen und realistische Werte zu erhalten.
- `--numjobs=1` : Der Vorgang wird von einem einzelnen Prozess (Thread) ausgeführt.

Test 2: Zufällige Lese- und Schreibzugriffe (Fokus auf IOPS)

Dieser Test ist der kritischste Teil des Vergleichs. Er simuliert das typische Verhalten von Betriebssystemen, virtuellen Maschinen oder relationalen Datenbanken, bei denen permanent winzige Datenmengen kreuz und quer auf der Festplatte gelesen und geändert werden.

- **--rw=randrw** : Definiert ein zufälliges Muster aus Lese- und Schreibzugriffen.
- **--bs=4k** : Setzt die Blockgröße auf 4 Kilobyte. Standard-Blockgröße der meisten Betriebssysteme.

Hier muss das Dateisystem maximalen Verwaltungsaufwand betreiben.

- **--size=500M** : Es werden 500 Megabyte an zufälligen Daten verarbeitet.
- **--numjobs=1** : Auch hier arbeitet ein einzelner Thread, um die reine Effizienz der Dateisystem-Architektur (Journaling bei ext4 vs. Copy-on-Write und Prüfsummen bei ZFS) messbar zu machen.

```
marvin@ubuntu-zfs:~$ ./run_benchmark.sh /benchpool
=====
Dateisystem Benchmark gestartet auf: /benchpool
=====
[ Test 1: Sequentielles Schreiben ]
-> Fokus: Durchsatz / Bandbreite (Simuliert große Dateien)
bw ( KIB/s): min=278528, max=993280, per=100.00%, avg=585728.00, stdev=367791.38, samples=3
iops      : min= 272, max= 970, avg=572.00, stdev=359.17, samples=3
WRITE: bw=534MiB/s (540MB/s), 534MiB/s-534MiB/s (540MB/s-540MB/s), io=1024MiB (1074MB), run=1917-1917msec

[ Test 2: Random Read/Write ]
-> Fokus: IOPS (Simuliert Datenbanken & OS-Zugriffe mit 4KB Blöcken)
bw ( KIB/s): min=10832, max=36048, per=99.93%, avg=20301.52, stdev=6433.60, samples=25
iops      : min= 2808, max= 9022, avg=5075.32, stdev=1688.45, samples=25
bw ( KIB/s): min=10796, max=36808, per=100.00%, avg=20220.68, stdev=6376.66, samples=25
iops      : min= 2764, max= 9202, avg=5055.16, stdev=1592.66, samples=25
READ:  bw=19.8MiB/s (20.7MB/s), 19.8MiB/s-19.8MiB/s (20.8MB/s-20.8MB/s), io=253MiB (263MB), run=12633-12633msec
WRITE: bw=19.7MiB/s (20.7MB/s), 19.7MiB/s-19.7MiB/s (20.7MB/s-20.7MB/s), io=249MiB (261MB), run=12633-12633msec

=====
Benchmark abgeschlossen! Bereit für die Dokumentation.
marvin@ubuntu-zfs:~$
```

```
marvin@ubuntu-ext4:~$ ./run_benchmark.sh ~/bench_ext4
=====
Dateisystem Benchmark gestartet auf: /home/marvin/bench_ext4
=====
[ Test 1: Sequentielles Schreiben ]
-> Fokus: Durchsatz / Bandbreite (Simuliert große Dateien)
bw ( KIB/s): min=778495, max=972752, per=100.00%, avg=870423.50, stdev=141603.08, samples=2
iops      : min= 752, max= 948, avg=850.00, stdev=138.59, samples=2
WRITE: bw=810MiB/s (849MB/s), 810MiB/s-810MiB/s (849MB/s-849MB/s), io=1024MiB (1074MB), run=1264-1264msec

[ Test 2: Random Read/Write ]
-> Fokus: IOPS (Simuliert Datenbanken & OS-Zugriffe mit 4KB Blöcken)
bw ( KIB/s): min=94512, max=96960, per=99.94%, avg=93462.40, stdev=1362.20, samples=5
iops      : min=24128, max=24974, avg=24615.60, stdev=340.55, samples=5
bw ( KIB/s): min=95192, max=100432, per=100.00%, avg=98116.80, stdev=2193.84, samples=5
iops      : min=23776, max=25108, avg=24529.20, stdev=548.46, samples=5
READ:  bw=96.2MiB/s (101MB/s), 96.2MiB/s-96.2MiB/s (101MB/s-101MB/s), io=253MiB (263MB), run=2605-2605msec
WRITE: bw=95.7MiB/s (100MB/s), 95.7MiB/s-95.7MiB/s (100MB/s-100MB/s), io=249MiB (261MB), run=2605-2605msec

=====
Benchmark abgeschlossen! Bereit für die Dokumentation.
marvin@ubuntu-ext4:~$
```

Die durchgeführten Leistungstests mit dem Werkzeug `fio` zeigen ein klares und architektonisch begründbares Leistungsgefälle zwischen den beiden Dateisystemen ext4 und ZFS.

Ergebnisse im sequentiellen Durchsatz (Test 1)

Beim sequentiellen Schreiben großer Datenmengen (1 Megabyte Blockgröße) erreichte das ext4-Dateisystem einen Durchsatz von 810 MiB/s, während ZFS mit 534 MiB/s messbar langsamer agierte. Das klassische Journaling-Dateisystem ext4 profitiert hier von seiner direkten, linearen Schreibmethode, während ZFS zusätzliche CPU-Zyklen für die Allokation neuer Blöcke und die Echtzeit-Berechnung von Prüfsummen benötigt.

Ergebnisse bei zufälligen Zugriffen (Test 2)

Der Stresstest mit zufälligen Lese- und Schreibzugriffen in 4-Kilobyte-Blöcken verdeutlicht die konzeptionellen Unterschiede am stärksten. Hier erreichte ext4 durchschnittlich 24.500 IOPS bei einer Bandbreite von knapp 96 MiB/s. ZFS lieferte im selben Szenario lediglich rund 5.000 IOPS bei knapp 20 MiB/s.

Fazit des Systemvergleichs

Die Messwerte belegen, dass ext4 in Szenarien, die auf maximale Rohleistung und hohe IOPS-Raten angewiesen sind, überlegen ist. ZFS erkaufte sich seine weitreichenden Enterprise-Funktionen (wie Copy-on-Write, Schutz vor unerkannter Datenkorruption und verlustfreie Snapshots) durch einen signifikanten Overhead. Besonders bei kleinen, zufälligen Schreibvorgängen führt der Zwang, Daten an neuen Orten zu speichern und Metadaten-Bäume inklusive Prüfsummen permanent neu aufzubauen, zu starken Leistungseinbußen gegenüber herkömmlichen "In-Place"-Dateisystemen.